Creating a New Table [Courses]

```
CREATE TABLE Courses
(
CourseId int Auto_increment Primary Key,
CourseName varchar(30) NOT NULL,
CourseDuration int not Null
course_Fee int not null);
```

Altering the Auto increment starting value

SYNTAX:

ALTER TABLE {table_name} AUTO_INCREMENT = {..}

Query:

```
ALTER TABLE Courses AUTO_INCREMENT = 5;
```

Creating a New Table [Students]

```
CREATE TABLE Students
(
StudentID INT AUTO_INCREMENT,
S_FN varchar(50) NOT NULL,
S_LN     "
S_Email  "      "      "
S_Phone  "
S_EnrollmentDate  timestamp  NOT NULL
```

timestamp

Selected Course int NOT NULL
(courseID)

YOE int NOT NULL

S_Company varchar(50) NOT NULL,

Batch_Start_Date $\boxed{\text{timestamp}}$ NOT NULL,

Location varchar(50) NOT NULL

PRIMARY KEY (Student_ID)

);

Adding a Constraints called UNIQUE for Col

ALTER TABLE Students {ADD} UNIQUE (email, phone);

We can add UNIQUE constraints for multiple col at a time but NOT NULL constraint cannot be added like this.

Adding a Constraint called NOT NULL for col

ALTER TABLE Students {MODIFY} S_FN varchar(40) NOT NULL;

ALTER TABLE Students {MODIFY} S_LN varchar(40) NOT NULL;

Creating a New Table Learners

```
CREATE TABLE Learners (
LearnerID INT AUTO_INCREMENT,
LFN VARCHAR (50),
LLN ,, ,,
L_Email varchar(50),
L_Phone varchar(50),
L_EnrollmentDate TIMESTAMP NOT NULL,
Selected_Course INT NOT NULL,
L_Company VARCHAR(50) NOT NULL,
L_SOJ varchar(50) ,, ,,
Batch_Start_Date TIMESTAMP NOT NULL,
Location varchar(50) NOT NULL,
PRIMARY KEY (LEARNERID)
UNIQUE(L_Email, L_Phone),
);
```

Primary key & Foreign key Constraints

✓ Primary key < Unique, NOT NULL

✓ Unique → Unique

✓ Foreign key

*I have only 15 Courses available currently → i.e only course-id

*If Learnes enter 16 as Selected_Course (course-id). Then the error occurs. To make this not happen. We use **FOREIGN KEY** Constraints

Foreign key (Selected Course) References Course (course-id) ← Parent table

*There can be multiple foreign key

Datatype Should be Same

TimeStamp : [DATATYPE]

Database store TimeStamp as Epoch as an Integer.

*Although it store as an integer. while retrieving from the db to the client side. It will be treated as DateTime
↳ converted

Range: [1970 - 01- 01 00:00:00 utc to
2038 - 01 - 19 03:14:07 utc]

↙
much lesser than DateTime

Datetime [5 byte]

→Use Datetime when you want to store usecase specific time (Eg. appointment, movie booking)

→Used to apply many DATE () function

→Human readable

~~Time~~ Timestamp [4 byte]

* Timestamp are stored as integer rep time in utc

→ timezone conversion happens during return & accepting.

return value changes with connection timezone

* Use timestamp when you want to record system time
  Eg: transactime

CHECK

Ensure that Batch_Start_Date should be after the L_Enrollment Date. So Alter the Constraints.

ALTER TABLE Learners ADD CHECK (Batch_Start_Date > L_Enrollment);

Data Analysis

[Employee], [Courses], [Learners]

Display the emp details, who is getting Maximum salary.

Query
```
SELECT * FROM employee
ORDER BY Salary DESC
LIMIT 1;
```

order of Execution

FROM → SELECT → ORDER BY → LIMIT

↑ (sort data)

for sorting the data, it should have data right?

Display the emp details who is getting Maximum salary with age>30

```
SELECT * FROM Employee
WHERE Age>30
ORDER BY Salary DESC
LIMIT 1.
```

OFE: From → Where → Select → Order by → Limit

WILD Cards

Query

Count the number of students who had enrolled in the month of Jan

SELECT count (*) from Learners
WHERE L_EnrollmentDate LIKE '____-01-%';

Alternate Way

SELECT count (*) from Learners
WHERE MONTH (L_EnrollmentDate) = 1;

Query

Update the YOE as 3 and L_Company as Amazon for Learner Rashmi and her ID is 104

UPDATE Learners
SET YOE = 3, L_Company = "Amazon"
WHERE LearnerID = 104